UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO
FACULTAD DE CIENCIAS DE LA COMPUTACIÓN / CIENCIAS DE LA INGENIERÍA
CARRERA DE INGENIERÍA DE SOFTWARE
ASIGNATURA: APLICACIONES DISTRIBUIDAS (SOF-I-R) — 7MO NIVEL "A"

ACTIVIDAD FORMATIVA: FORM-FB-01

Fichas Bibliográficas: Fundamentos de Sistemas Distribuidos y Arquitecturas Distribuidas

1. Datos Informativos de la Actividad

Estudiante Responsable:	Kenny Isaías España Vera (**C.I.:** 1208867794)
Correo Institucional:	`kespanav@uteq.edu.ec`
Docente Titular:	Dr. Gleiston Cicerón Guerrero Ulloa
Período Académico:	Semana 2

2. Introducción y Alcance de la Actividad Formativa

El diseño y mantenimiento de arquitecturas de microservicios demanda una comprensión profunda de cómo se dividen las responsabilidades y de su impacto en el rendimiento bajo entornos distribuidos reales. Esta actividad formativa individual (**FORM-FB-01**) tiene como propósito establecer el marco bibliográfico preparatorio para el informe sumativo **TA-IND-01**, donde se llevará a cabo un análisis crítico comparativo de un sistema distribuido real fundamentado en los conceptos de la Unidad 1 (transparencias de distribución, modelos de comunicación, tolerancia a fallos y los teoremas CAP/PACELC).

Para este compendio se examinaron y verificaron cinco fuentes académicas recientes (publicadas entre 2023 y 2024 en revistas Q1 como *IEEE Access* y *Journal of Systems and Software*, además de la conferencia internacional *IEEE ICSA*). Las obras abordan problemáticas complementarias: la vinculación entre los principios clásicos de modularidad y la descomposición moderna, la evolución técnica de los repositorios de código abierto, la medición empírica de sobrecargas en clústeres físicos de Kubernetes, el modelado formal con redes de colas y la identificación en tiempo de ejecución de antipatrones dinámicos.

Con la finalidad de ofrecer una estructura clara y un valor agregado analítico, cada fuente se presenta mediante una matriz tabular individual con bordes definidos. Este formato organiza sistemáticamente la referencia formal en estándar IEEE, su identificador persistente comprobado, un resumen analítico del texto, un dato empírico resumido con su debida contextualización y una nota personal que delimita su aplicación directa en el informe final.

3. Ficha Bibliográfica 1: Principios y Patrones en la Descomposición de Microservicios

FICHA DE LECTURA N.° 1: PRINCIPIOS Y PATRONES	
Cita	V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: 10.1109/ACCESS.2023.3305687
DOI	10.1109/ACCESS.2023.3305687 (Verificado en IEEE Xplore).
Resumen Analítico	La transición desde aplicaciones monolíticas hacia microservicios suele fracasar cuando los equipos desconocen los principios fundamentales que deben guiar la partición del software. Este trabajo presenta un estudio exhaustivo que conecta la teoría de descomposición modular de David Parnas (1972) con las pautas modernas de Martin Fowler y Sam Newman. Los autores evalúan patrones clásicos como Strangler Fig, Backend for Frontend (BFF), Database per Service y Aggregator, analizando sus implicaciones prácticas. Concluyen que la independencia real de los microservicios solo se alcanza si se respeta el ocultamiento de información en los límites de negocio, advirtiendo que compartir bases de datos degenera en antipatrones severos.
Dato Concreto	**Dato (Sección V, Tabla 2, pág. 88349):** Mapeo de modularidad: el criterio de Parnas (1972) de reemplazar módulos de forma independiente equivale a la componentización por servicios de Fowler y al ocultamiento de detalles internos de Newman. En fallos, Fowler plantea *Design for Failure* y Newman *Isolate Failure*. **Explicación:** La partición en microservicios retoma la encapsulación clásica de hace 50 años. Compartir bases de datos entre servicios destruye este aislamiento y recrea dependencias monolíticas.
Nota Personal	Este artículo me ayudó a comprender que diseñar microservicios no es simplemente levantar contenedores separados, sino definir fronteras de información muy estrictas. Para el informe TA-IND-01, utilizaré esta fuente en la sección de Arquitectura del Sistema y en el análisis de las Transparencias de Distribución (acceso y ubicación), como base para evaluar si el sistema distribuido seleccionado realmente encapsula sus datos o si mantiene dependencias ocultas entre componentes.

4. Ficha Bibliográfica 2: Análisis Empírico de la Coevolución en Repositorios de Microservicios

FICHA DE LECTURA N.° 2: COEVOLUCIÓN EN MICROSERVICIOS	
Cita	W. K. G. Assunção et al., «How do microservices evolve? An empirical analysis of changes in open-source microservice repositories,» *Journal of Systems and Software*, vol. 204, pág. 111 788, 2023, **artno** 111788. DOI: 10.1016/j.jss.2023.111788
DOI	10.1016/j.jss.2023.111788 (Verificado en Elsevier ScienceDirect).
Resumen Analítico	A pesar de que la literatura promueve la total autonomía e independencia de despliegue de los microservicios, existe escasa evidencia empírica sobre cómo evolucionan en proyectos reales. Los autores desarrollan un estudio a gran escala examinando 7.319 commits en 11 repositorios de código abierto, combinando análisis cuantitativo de cambios atómicos con inspección cualitativa. Sus hallazgos demuestran que el mantenimiento está dominado por requerimientos técnicos (infraestructura, librerías compartidas y automatización) más que por reglas de negocio. Asimismo, detectaron patrones recurrentes de coevolución, tales como commits de tipo *shotgun surgery*, donde la modificación de una funcionalidad fuerza alteraciones simultáneas y coordinadas en múltiples microservicios del sistema.
Dato Concreto	**Dato (Sección 4.1, Tabla 2, pág. 7):** En 7.319 commits de 11 proyectos, más del 50 % (y hasta el 70 % en sistemas complejos) corresponde solo a cambios técnicos (infraestructura y librerías), mientras que los cambios exclusivos de lógica de negocio promedian apenas entre el 2 % y el 15 %. **Explicación:** Los microservicios demandan más esfuerzo de mantenimiento técnico (red, dependencias y despliegue) que el desarrollo de funciones de negocio puras.
Nota Personal	Me pareció una lectura muy aterrizada a la realidad porque desmitifica la supuesta independencia total de los servicios. En el informe TA-IND-01, emplearé esta referencia en la sección de Postura Crítica (compromisos y trade-offs) y en el apartado de Tolerancia a Fallos, para justificar cómo el acoplamiento técnico en protocolos y librerías compartidas puede desencadenar fallos de compatibilidad en cascada ante actualizaciones imprevistas.

5. Ficha Bibliográfica 3: Evaluación Experimental de Rendimiento en Clústeres Reales

FICHA DE LECTURA N.° 3: RENDIMIENTO EXPERIMENTAL	
Cita	W. Meijer et al., «Experimental evaluation of architectural software performance design patterns in microservices,» *Journal of Systems and Software*, vol. 218, pág. 112 183, dic. de 2024, **artno** 112183. DOI: 10.1016/j.jss.2024.112183
DOI	10.1016/j.jss.2024.112183 (Verificado en Elsevier ScienceDirect).
Resumen Analítico	Las decisiones de diseño arquitectónico en sistemas distribuidos suelen apoyarse en modelos analíticos abstractos que no capturan el comportamiento real bajo estrés. Para verificar su precisión, los autores implementaron un entorno experimental con Kubernetes en una nube privada, ejecutando pruebas de carga de 5 minutos con más de 9.000 peticiones en cuatro patrones: Gateway Aggregator, Gateway Offloading, Chain of Responsibility y CQRS. Los resultados revelan que, si bien las predicciones teóricas guardan correlación con la tendencia general ($r > 0.8$, $p < 0.001$), subestiman los tiempos de respuesta reales debido a sobrecargas de serialización, desempaquetado de mensajes y latencia de red en nodos físicos.
Dato Concreto	**Dato (Sección 4.3, Tabla 4, págs. 9–11):** En clúster Kubernetes real, los modelos teóricos subestiman la latencia entre un 10.62 % y más del 15 %; además, en Gateway Aggregation el desempaquetado de mensajes causó saturación no lineal de CPU en el nodo puerta de enlace. **Explicación:** La deserialización y ensamblado de respuestas en hardware físico genera demoras reales de CPU que los modelos ideales de rendimiento suelen ignorar.
Nota Personal	Me gustó este artículo porque aporta rigor empírico y demuestra que el rendimiento real no se calcula solo con fórmulas ideales. Para el informe TA-IND-01, usaré esta fuente en la sección de Experimentación Propia y en el análisis de CAP y PACELC (latencia frente a consistencia), para respaldar las mediciones de tiempos de respuesta del sistema y justificar técnicamente los cuellos de botella detectados en la red.

6. Ficha Bibliográfica 4: Modelado Analítico de Patrones mediante Redes de Colas

FICHA DE LECTURA N.° 4: MODELADO CON REDES DE COLAS	
Cita	R. Pinciroli et al., «Performance Modeling and Analysis of Design Patterns for Microservice Systems,» en *2023 IEEE 20th International Conference on Software Architecture (ICSA)*, 2023, págs. 35-46. DOI: 10.1109/ICSA56044.2023.00012
DOI	10.1109/ICSA56044.2023.00012 (Verificado en IEEE Xplore).
Resumen Analítico	Evaluar el impacto de adoptar patrones de diseño en microservicios antes de su despliegue operativo es un reto complejo para los arquitectos de software. Este artículo formaliza matemáticamente siete patrones arquitectónicos de uso industrial (Anti-Corruption Layer, BFF, CQRS, Competing Consumers, Gateway Aggregation, Gateway Offloading y Static Content Hosting) mediante Redes de Colas (Queueing Networks) y Redes de Petri híbridas. Mediante simulaciones bajo cargas de trabajo heterogéneas, los autores analizan las ganancias y penalizaciones de rendimiento (*pains and gains*). Demuestran que descargar funciones transversales al Gateway (Gateway Offloading) puede generar graves retrasos de latencia si la carga supera la capacidad del intermediario.
Dato Concreto	**Dato (Sección IV-B, Fig. 5, págs. 41–43):** Con redes de colas se comprobó que si las tareas descargadas en Gateway Offloading superan el 50 % de la carga, la cola del Gateway se satura y la latencia promedio se incrementa más del 40 % respecto al diseño sin descarga. **Explicación:** Concentrar demasiadas tareas (seguridad, cuotas, registro) en el Gateway convierte la entrada en un embotellamiento crítico.
Nota Personal	Este texto me enseñó que los patrones arquitectónicos no son balas de plata y que traen consigo penalizaciones de rendimiento calculables. En el informe TA-IND-01, voy a utilizar este artículo en la sección de Modelos de Comunicación y en el Análisis Comparativo, para confrontar el impacto del encolamiento y los intermediarios centralizados frente a esquemas de mensajería asíncrona punto a punto.

7. Ficha Bibliográfica 5: Visualización Dinámica de Antipatrones en Tiempo de Ejecución

FICHA DE LECTURA N.° 5: VISUALIZACIÓN DE ANTIPATRONES	
Cita	G. Parker et al., «Visualizing Anti-Patterns in Microservices at Runtime: A Systematic Mapping Study,» *IEEE Access*, vol. 11, págs. 4434-4442, 2023. DOI: 10.1109/ACCESS.2023.3236165
DOI	10.1109/ACCESS.2023.3236165 (Verificado en IEEE Xplore).
Resumen Analítico	La complejidad topológica de las arquitecturas de microservicios dificulta la detección de anomalías estructurales mediante análisis estático de código fuente, ya que muchos antipatrones solo se manifiestan bajo el tráfico real de producción. Este estudio de mapeo sistemático analiza las herramientas y métodos existentes para identificar y visualizar antipatrones dinámicos (dependencias cíclicas, cuellos de botella y nudos o *knots*). Los resultados evidencian que las dependencias cíclicas en tiempo de ejecución son las más reportadas y riesgosas al inducir bloqueos y sobrecarga en cascada. Los autores destacan que la integración de trazas distribuidas con bases de datos de grafos resulta fundamental para diagnosticar el sistema.
Dato Concreto	**Dato (Sección IV, Tabla 2, págs. 4438–4440):** Las dependencias cíclicas abarcan más del 60 % de detecciones en grafos de servicios, pero problemas dinámicos severos como nudos (*knots*) o cuellos de botella en runtime solo son detectados por el 20 % de las herramientas analizadas. **Explicación:** Los servicios pueden parecer desacoplados en el código estático, pero en ejecución generan bucles circulares de llamadas que saturan la memoria y bloquean el sistema.
Nota Personal	Me pareció una investigación fundamental porque demuestra que revisar el repositorio no basta para garantizar la estabilidad de un sistema distribuido. En el informe TA-IND-01, incorporaré esta fuente en la sección de Tolerancia a Fallos (detección y recuperación de incidentes) y en la Postura Crítica, para analizar si el sistema distribuido elegido provee observabilidad con grafos de dependencia o si es susceptible a fallas circulares en runtime.

8. Declaración de Uso Ético de Inteligencia Artificial

De acuerdo con las pautas de la actividad académica, se declara de forma transparente el uso de herramientas de inteligencia artificial en la preparación de este trabajo:

- **Herramienta y alcance:** Se empleó un modelo de lenguaje exclusivamente como soporte para la diagramación de tablas matriciales en LaTeX (*tabularx*) y en la verificación de la sintaxis del archivo de bibliografía con Biber.
- **Verificación de fuentes:** Todas las fuentes académicas citadas fueron consultadas directamente desde los artículos originales en formato PDF. Se comprobó manualmente que los enlaces DOI correspondan con exactitud a los registros oficiales en IEEE Xplore y Elsevier ScienceDirect.
- **Autoría y redacción:** Los resúmenes, los datos concretos seleccionados y las notas personales sobre la aplicación en el informe TA-IND-01 fueron redactados de forma individual a partir de la lectura y comprensión propia de los documentos, evitando expresiones genéricas o textos automáticos.

Referencias Bibliográficas (Formato IEEE)

[1] V. Velepucha y P. Flores, «A Survey on Microservices Architecture: Principles, Patterns and Migration Challenges,» *IEEE Access*, vol. 11, págs. 88 339-88 358, 2023. DOI: 10.1109/ACCESS.2023.3305687

[2] W. K. G. Assunção, J. Krüger, S. Mosser y S. Selaoui, «How do microservices evolve? An empirical analysis of changes in open-source microservice repositories,» *Journal of Systems and Software*, vol. 204, pág. 111 788, 2023, **artno** 111788. DOI: 10.1016/j.jss.2023.111788

[3] W. Meijer, C. Trubiani y A. Aleti, «Experimental evaluation of architectural software performance design patterns in microservices,» *Journal of Systems and Software*, vol. 218, pág. 112 183, dic. de 2024, **artno** 112183. DOI: 10.1016/j.jss.2024.112183

[4] R. Pinciroli, A. Aleti y C. Trubiani, «Performance Modeling and Analysis of Design Patterns for Microservice Systems,» en *2023 IEEE 20th International Conference on Software Architecture (ICSA)*, 2023, págs. 35-46. DOI: 10.1109/ICSA56044.2023.00012

[5] G. Parker et al., «Visualizing Anti-Patterns in Microservices at Runtime: A Systematic Mapping Study,» *IEEE Access*, vol. 11, págs. 4434-4442, 2023. DOI: 10.1109/ACCESS.2023.3236165